

AI Report

We predict this text is

Human Generated

AI Probability

18%

This number is the probability that the document is AI generated, not a percentage of AI text in the document.

Plagiarism



The plagiarism scan was not run for this document. Go to gptzero.me to check for plagiarism.

ece (3).pdf - 5/17/2026

TGR-LORA: A CONTINUAL LEARNING PIPELINE FOR SMALL LANGUAGE MODELS

A Design Project Report Presented to the School of Electrical and Computer Engineering of Cornell University in Partial Fulfillment of the Requirements for the Degree of Master of Engineering, Electrical and Computer Engineering

Submitted by

Beichen Yu (by325)

MEng Field Advisor: Adit Jain (aj457@cornell.edu), Vikram Krishnamurthy
(vikramk@cornell.edu)

Degree Date: December 2026

Abstract

Master of Engineering Program

School of Electrical and Computer Engineering

Cornell University

Design Project Report

Project Title: TGR-LoRA: A CONTINUAL LEARNING PIPELINE FOR SMALL LANGUAGE MODELS

Author: Beichen Yu (by325)

Abstract

: This project implements and evaluates a parameter-efficient (PEFT) continual learning (CL) method designed for small language models (SLMs). It aims to address the problem of catastrophic forgetting, which means during training process across a sequence of different tasks, the parameter changes for learning later tasks might degrade the model's performance on previous tasks. Our project utilizes the Qwen2.5-3B-Instruct model and conducts experiments using five subsets from the TRACE continual learning enchmark, including C-STANCE, FOMC, ScienceQA, NumGLUE-cm, and NumGLUE-ds.

We implemented two distinct methods. Sequential LoRA serves as our baseline. It freezes the base model parameters and trains a single LoRA adapter across all tasks.

TGR/OrthoHist-LoRA is our method, which trains an active LoRA branch for the current task while integrating and consolidating the updated parameters into a frozen pool of historical bases and coefficients. In order to effectively use historical information, it also learns a low-rank mixing operator (defined as $M_t = I + U_t V_t^{\top}$) for the current task, thereby enabling the reuse of historical activation information. Furthermore, we implemented the metrics for evaluating continual learning performance, visualizations such as heatmaps showing knowledge reuse degree.

Executive Summary

The main objective of our project was to mitigate catastrophic forgetting while enabling subsequent tasks to leverage useful information acquired from prior tasks. Our system is entirely software-based. The core components include the Qwen2.5-3B-Instruct model, LoRAbased PEFT methods, TGR-LoRA method, and the TRACE benchmark.

Our proposed method involves training an Active LoRA branch tailored to the current task. Updated parameters from completed tasks are stored in a frozen historical pool, and each new task is equipped with a trainable low-rank mixing operator for reusing these historical activations. During the training process, both the base model and the historical pool remain frozen. There are 2 benefits: preventing the new training process from directly overwriting previously learned directions, while allowing subsequent tasks to reuse knowledge acquired during earlier tasks at the same time. In summary, the design philosophy is that the Active LoRA branch facilitates the model's exploration of new knowledge of the current task, while the low-rank mixing operator enables the efficient reuse of historical knowledge.

We successfully implemented a comprehensive, task-by-task training loop with checkpointbased resume, task-level JSON logging, score matrix generation, Backward Transfer (BWT) calculation, average accuracy (AA) computation, and visualizations such as feature reuse heatmaps for the historical mixing operators.

During the project's initial poster presentation phase, reported results indicated more pronounced positive BWT scores than those subsequently reproduced in the final implementation. During the final code integration and validation phase, we refined our evaluation method, using to a method that computes task-by-task score matrices and BWT metrics. Under this more rigorous evaluation standard, we further refined our experimental results and drew the following conclusions:

Individual Contributions

Beichen Yu (by325): This is an individual project. I was responsible for the project's overall design, code implementation, debugging, experimental execution, poster creation, code repository organization, and final report writing. My specific contributions include:

Defined the Continual Learning problem setting, selecting the Qwen2.5-3B-Instruct model and the TRACE benchmark as the foundation for experimental validation.

Implemented a Sequential LoRA baseline model using the PEFT library, freezing the parameters of the Base Model during the implementation process.

Implemented the TGR-LoRA layer wrapper module. The components include the frozen base model, an active LoRA branch, a frozen pool of historical basis and coefficients, an orthogonal gradient projection mechanism, and a low-rank mixer $M_t = I + U_t V_t^{\text{top}}$, etc.

Constructed a Task-by-Task Trainer, checkpoint saving logic, training resume logic, JSON logging function, a score matrix saving function, and a module for analyzing historical knowledge reuse.

Implemented task-specific answer parsing functions and text generation evaluation metrics for five datasets: C-STANCE, FOMC, ScienceQA, NumGLUE-cm, and NumGLUE-ds to ensure an accurate evaluation method.

Executed and debugged various experiments within Colab and AutoDL GPU environments, resolving technical challenges such as PyTorch/Transformers compatibility issues, CUDA architecture adaptation problems, and RTX 4090 GPU memory limitations.

Organized and published the final GitHub code repository including the README and configuration files, and made detailed documentation to ensure the reproducibility of the experiments.

Contents

Abstract i

Executive Summary ii

Individual Contributions iii

1 Problem Statement and Requirements 1

1.1 Design Problem 1

1.2 Requirements 1

1.3 Practical Constraints 2

1.4 Final Design 3

2 Review of Alternatives and Selected Approach 4

2.1 Full Fine-Tuning 4

2.2 Sequential LoRA 4

2.3 Adapter-per-Task 5

2.4 Fisher-SVD Design 5

2.5 Final Selected Approach: TGR/OrthoHist-LoRA 5

3 System Design and Implementation 9

3.1 System Overview	9
3.2 Repository Structure	9
3.3 Sequential LoRA Baseline Implementation	10
3.4 TGR-LoRA Implementation	10
4 Testing and Experimental Validation	12
4.1 Evaluation Method	12
4.2 Sequential LoRA Five-Task Baseline	13
4.3 TGR-LoRA Five-Task Comparison	13
4.3.1 TGR-LoRA with $m_t \setminus k=4$	14
4.3.2 TGR-LoRA with $m_t \setminus k=8$	14
4.3.3 TGR-LoRA with $m_t \setminus k=16$	15
4.3.4 Final Method Comparison	15
5 Reproducibility, User Manual, and Bill of Materials	17
5.1 Code Repository	17
5.2 Environment Setup	17
5.3 Dataset Layout	17
5.4 Running Experiments	18
5.5 Expected Outputs	18
5.6 Software Bill of Materials	19
A Appendix A: Code Listings	21
A. 1 Project Entry Point	21
A. 2 Experiment Runner	23
A. 3 Result Summary Script	24
A. 4 Python Requirements	27
A. 5 Configuration: Sequential LoRA	27
A. 6 Configuration: OrthoHist-LoRA $\mathrm{k}=4$	28
A. 7 Configuration: OrthoHist-LoRA $\mathrm{k}=8$	29
A. 8 Configuration: OrthoHist-LoRA $\mathrm{k}=16$	30

A. 9 Configuration Loader	31
A. 10 TRACE Data Loader	32
A. 11 TRACE Evaluation Utilities	36
A. 12 Training Loops	45
A. 13 OrthoHist-LoRA Implementation	58
A. 14 Checkpointing Utilities	63
A. 15 Analysis and Visualization Utilities	64
B Appendix B: User Manual	67
B. 1 Quick Start	67
B. 2 Reading Final Metrics	67

Chapter 1

Problem Statement and Requirements

1.1 Design Problem

Continual Learning aims to enable models to train on a continuous stream of incoming tasks. While learning a current task, a model must maintain its performance on previously learned tasks without degradation. Large Language Models (LLMs) are very susceptible to this challenge, as gradient updates applied to subsequent tasks may overwrite specific directions of model parameters that are critical for earlier tasks. The direct consequence of that is Catastrophic Forgetting.

This project focuses on PEFT supervised fine-tuning methods and Small Language Models (SLMs), as these methods are more lightweight and easier to deploy in real-world scenarios. We selected LoRA, as it achieves excellent results by freezing the base model and training only a set of low-rank adapter matrices. However, if a single LoRA adapter is trained sequentially across tasks, the model may still struggle to avoid the forgetting problem. While training a separate adapter for each task effectively prevents interference between tasks, this approach suffers from extremely low parameter reuse. Furthermore, during the inference phase, the corresponding adapter must be manually selected for each specific task, which can be complex to manage. Consequently, the core design challenge of this project is constructing a dynamically growing continual learning system that can both effectively preserve information of historical tasks and enable the efficient reuse of that information.

1.2 Requirements

The final requirements evolved iteratively over the two semesters. The focus of the first semester was to investigate the causes of catastrophic forgetting. Through a comprehensive literature review, we knew that catastrophic forgetting is most severe when the input spaces are similar but the output spaces differ. The work in the second semester focused on designing a robust training and evaluation pipeline with mathematical rigor. The finalized requirements are as follows:

Sequential Task Training: The system must train tasks sequentially in a fixed order, performing an evaluation after the completion of each task.

Parameter Efficiency: The Base Model must remain frozen, and only parameters associated with LoRA or TGR-LoRA are permitted to be trained.

Baseline

● Sentences that are likely AI-generated.

FAQs

What is GPTZero?

GPTZero is the leading AI detector for checking whether a document was written by a large language model such as ChatGPT. GPTZero detects AI on sentence, paragraph, and document level. Our model was trained on a large, diverse corpus of human-written and AI-generated text with support for English, Spanish, French, German, and other languages. To date, GPTZero has served over 17 million users around the world, and works with over 100 organizations in education, hiring, publishing, legal, and more.

When should I use GPTZero?

Our users have seen the use of AI-generated text proliferate into education, certification, hiring and recruitment, social writing platforms, disinformation, and beyond. We've created GPTZero as a tool to highlight the possible use of AI in writing text. In particular, we focus on classifying AI use in prose. Overall, our classifier is intended to be used to flag situations in which a conversation can be started (for example, between educators and students) to drive further inquiry and spread awareness of the risks of using AI in written work.

Does GPTZero only detect ChatGPT outputs?

No, GPTZero works robustly across a range of AI language models, including but not limited to ChatGPT, GPT-5, GPT-4, GPT-3, Gemini, Claude, and AI services based on those models.

What are the limitations of the classifier?

The nature of AI-generated content is changing constantly. As such, these results should not be used to punish students. We recommend educators to use our behind-the-scenes [Writing Reports](#) as part of a holistic assessment of student work. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Instead, we recommend educators take approaches that give students the opportunity to demonstrate their understanding in a controlled environment and craft assignments that cannot be solved with AI. Our classifier is not trained to identify AI-generated text after it has been heavily modified after generation (although we estimate this is a minority of the uses for AI-generation at the moment). Currently, our classifier can sometimes flag other machine-generated or highly procedural text as AI-generated, and as such, should be used on more descriptive portions of text.

I'm an educator who has found AI-generated text by my students. What do I do?

Firstly, at GPTZero, we don't believe that any AI detector is perfect. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Nonetheless, we recommend that educators can do the following when they get a positive detection: Ask students to demonstrate their understanding in a controlled environment, whether that is through an in-person assessment, or through an editor that can track their edit history (for instance, using our [Writing Reports](#) through Google Docs). Check out our list of [several recommendations](#) on types of assignments that are difficult to solve with AI.

Ask the student if they can produce artifacts of their writing process, whether it is drafts, revision histories, or brainstorming notes. For example, if the editor they used to write the text has an edit history (such as Google Docs), and it was typed out with several edits over a reasonable period of time, it is likely the student work is authentic. You can use GPTZero's Writing Reports to replay the student's writing process, and view signals that indicate the authenticity of the work.

See if there is a history of AI-generated text in the student's work. We recommend looking for a long-term pattern of AI use, as opposed to a single instance, in order to determine whether the student is using AI.